

Wavepacket in a Potential Box

In this section, we will explore the time evolution of a wavepacket confined in a potential box with infinite walls. This builds directly on our understanding of stationary states from previous lectures, extending it to show how quantum states evolve dynamically over time.

```
#!/ edit: false
#!/ echo: false
#!/ execute: true
import numpy as np
import matplotlib.pyplot as plt
from scipy.constants import *
from scipy.sparse import diags
from scipy.sparse.linalg import eigsh
from scipy.fftpack import fft,ifft
from time import sleep,time

# Physical constants (for this simulation, we'll later set  $\hbar=1$  and  $m=1$ )
hbar = 1.0545718e-34 # Reduced Planck constant in J·s
electron_mass = 9.10938356e-31 # Electron mass in kg

# default values for plotting
plt.rcParams.update({
    'font.size': 12,
    'lines.linewidth': 1,
    'lines.markersize': 5,
    'axes.labelsize': 11,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'xtick.top': True,
    'xtick.direction': 'in',
    'ytick.right': True,
    'ytick.direction': 'in',
})
```

```
def get_size(w, h):  
    return (w/2.54, h/2.54)
```

Problem Setup

We now want to study the dynamics of a particle in a box, extending our previous stationary analysis to understand how a wavepacket evolves over time when confined between two impenetrable walls.

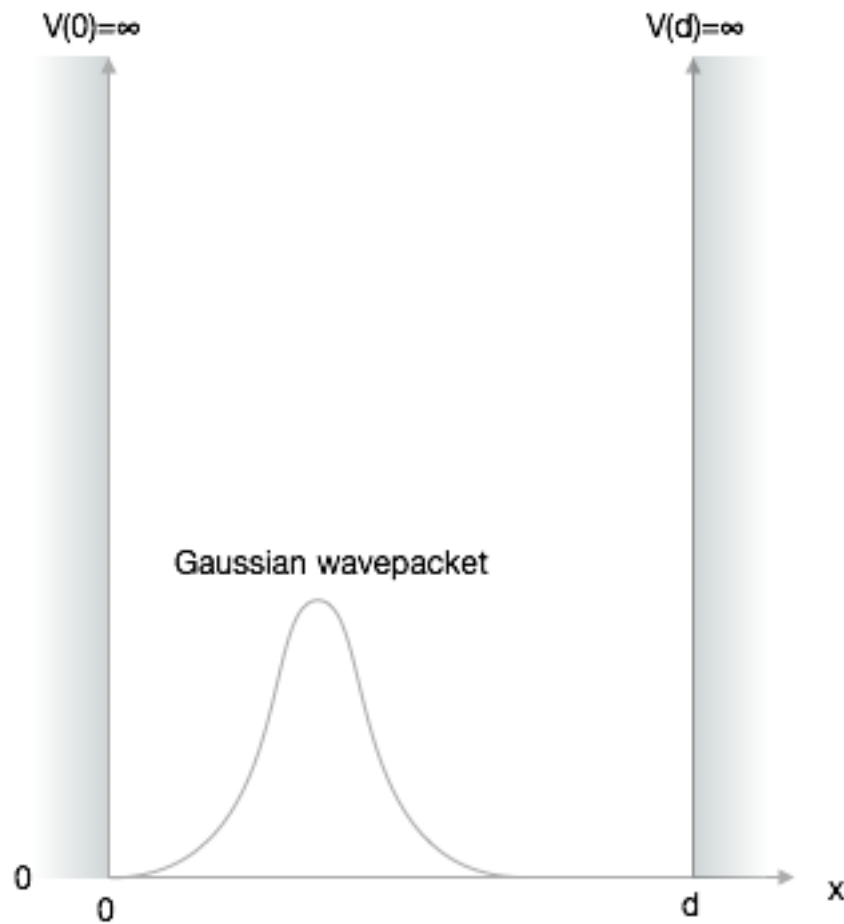


Figure 1: Particle_In_Box.png

For a potential box with infinite walls and width d , the potential energy inside the box is zero:

$$V(x) = \begin{cases} 0, & 0 < x < d \\ \infty, & \text{otherwise} \end{cases}$$

The time-dependent Schrödinger equation within the box therefore reduces to:

$$-i\hbar \frac{\partial \Psi}{\partial t} = -\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} \Psi \quad (1)$$

Time-Dependent Solution Using Stationary States

Stationary States as a Basis

From our previous work, we know the stationary solutions (energy eigenstates) for this system are standing waves:

$$\psi_n(x) = \sqrt{\frac{2}{d}} \sin\left(\frac{n\pi}{d}x\right), \quad n = 1, 2, 3, \dots \quad (2)$$

with corresponding energy eigenvalues:

$$E_n = \frac{\hbar^2 \pi^2 n^2}{2md^2} \quad (3)$$

These stationary states form a complete orthonormal basis, meaning any wavefunction that satisfies the boundary conditions can be expressed as a linear combination of these states. This is how we'll construct our time-dependent solution.

Constructing the Time-Dependent Solution

The general time-dependent solution can be written as a superposition of the stationary states, each evolving with its own time-dependent phase factor:

$$\Psi(x, t) = \sum_{n=1}^{\infty} C_n(t) \sqrt{\frac{2}{d}} \sin\left(\frac{n\pi}{d}x\right) \quad (4)$$

The sine function ensures that the boundary conditions are satisfied: $\Psi(x = 0, t) = 0$ and $\Psi(x = d, t) = 0$. The time dependence is contained in the coefficients $C_n(t)$.

To find these coefficients, we substitute this solution into the time-dependent Schrödinger equation:

$$i\hbar \sum_{n=1}^{\infty} \frac{dC_n(t)}{dt} \sqrt{\frac{2}{d}} \sin\left(\frac{n\pi}{d}x\right) = \frac{\hbar^2}{2m} \sum_{n=1}^{\infty} C_n(t) \sqrt{\frac{2}{d}} \frac{n^2\pi^2}{d^2} \sin\left(\frac{n\pi}{d}x\right) \quad (5)$$

Solving for the Time-Dependent Coefficients

To obtain a differential equation for each coefficient $C_n(t)$, we multiply both sides by $\sin\left(\frac{l\pi}{d}x\right)$ and integrate from 0 to d . Due to the orthogonality of the sine functions:

$$\int_0^d \sin\left(\frac{n\pi}{d}x\right) \sin\left(\frac{l\pi}{d}x\right) dx = \frac{d}{2} \delta_{nl} \quad (6)$$

where δ_{nl} is the Kronecker delta (equals 1 when $n = l$, and 0 otherwise).

This gives us a separate differential equation for each coefficient:

$$\frac{dC_n(t)}{dt} = -i \frac{E_n}{\hbar} C_n(t) = -i \frac{\hbar\pi^2 n^2}{2md^2} C_n(t) \quad (7)$$

The solution to this first-order differential equation is:

$$C_n(t) = C_n(0) e^{-i \frac{E_n}{\hbar} t} = C_n(0) e^{-i \frac{\hbar\pi^2 n^2}{2md^2} t} \quad (8)$$

where $C_n(0)$ are the initial coefficients determined by the wavepacket's shape at $t = 0$.

Complete Time-Dependent Solution

Thus, our complete time-dependent solution is:

$$\Psi(x, t) = \sum_{n=1}^{\infty} C_n(0) \sqrt{\frac{2}{d}} e^{-i \frac{\hbar\pi^2 n^2}{2md^2} t} \sin\left(\frac{n\pi}{d}x\right) \quad (9)$$

The physical interpretation is that each energy eigenstate evolves with its own characteristic frequency $\omega_n = E_n/\hbar$, and their interference creates the time-evolving wavepacket.

Initial Conditions: Gaussian Wavepacket

For our simulation, we'll use a Gaussian wavepacket as the initial state. While this doesn't strictly satisfy the boundary conditions at $t = 0$ (the Gaussian extends beyond the box), we'll choose parameters so that the wavefunction is effectively zero at the boundaries.

For simplicity, we'll use natural units where $\hbar = 1$ and $m = 1$, and we'll set the box width to $d = \pi$.

Here's a function to create a Gaussian wavepacket:

```
#| autorun: false
def gauss_x(x, sigma, x0, k0):
    # Create the unnormalized wavepacket
    tmp = np.exp(-0.5 * ((x - x0)/ sigma) ** 2 + 1j * x * k0)/(sigma * np.sqrt(np.pi))

    # Normalize the wavefunction
    norm = np.sum(np.abs(tmp)**2)
    return (tmp/np.sqrt(norm))
```

Let's visualize this initial wavepacket:

```
#| autorun: false
xp = np.linspace(0, np.pi, 1000) # Position grid from 0 to

plt.figure(figsize=get_size(10, 6))
plt.plot(xp, np.abs(gauss_x(xp, 0.1, np.pi/4, 0))**2)
plt.xlabel('Position x')
plt.ylabel(r'Probability density  $|\Psi(x)|^2$ ')
plt.axvline(x=0, ls='--', color='k', label='Box boundaries')
plt.axvline(x=np.pi, ls='--', color='k')

plt.tight_layout()
plt.show()
```

Expressing the Initial State in the Energy Basis

To simulate the time evolution, we need to express our initial Gaussian wavepacket in terms of the energy eigenstates. First, let's define a function to generate the energy eigenfunctions:

```
#| autorun: false
def eigen_f(x, n, d):
    return np.sin(n*np.pi*x/d)*np.sqrt(2/d)
```

To find the expansion coefficients $C_n(0)$, we need to calculate the inner product:

$$C_n(0) = \int_0^d \Psi(x, 0) \psi_n(x) dx \quad (10)$$

Let's implement this using numerical integration:

```
#| autorun: false
def integrate(fun, a, b, sigma, x0, k0, *params):
    N = 10000 # Number of integration points
    x = np.linspace(a, b, N)
    fu = fun(x, sigma, x0, k0)*eigen_f(x, *params)
    return np.sum(fu)*(b-a)/N
```

Now we can calculate the initial coefficients for a certain number of eigenfunctions:

```
#| autorun: false
def init(num_states):
    c0n = np.zeros(num_states+1, dtype=complex) # +1 because we start counting from n=1

    # Skip n=0 (not a valid state)
    for n in range(1, len(c0n)):
        c0n[n] = integrate(gauss_x, 0, np.pi, 0.1, np.pi/4, 0, n, np.pi)

    return c0n
```

Let's calculate the coefficients for the first 30 eigenstates:

```
#| autorun: false
# Calculate coefficients for 30 eigenstates
c0n = init(30)
```

Time Evolution Function

Now we can implement the time evolution of our wavepacket:

```
#| autorun: false
def w_func(x, d, c0n, t):

    psi = 0
```

```

for n in range(1, len(c0n)): # Start from n=1
    # Phase factor from time evolution
    phase = np.exp(-1j*(n**2*np.pi**2)*t/(2*d**2))
    psi = psi + c0n[n]*phase*eigen_f(x, n, d)
return psi

```

Let's verify that our expansion correctly reproduces the initial wavepacket:

```

#| autorun: false
xp = np.linspace(0, np.pi, 1000)
wv = w_func(xp, np.pi, c0n, 0) # Wavefunction at t=0

plt.figure(figsize=get_size(10, 6))
plt.plot(xp, 10*np.abs(wv)**2, label='Eigenstate expansion') # Scale factor of 10 for proper
plt.plot(xp, np.abs(gauss_x(xp, 0.1, np.pi/4, 0))**2, 'k--', label='Exact Gaussian')

plt.xlabel('Position x')
plt.ylabel(r'Probability density  $|\Psi(x)|^2$ ')

plt.tight_layout()
plt.show()

```

Quantum Time Evolution: Animation

Now we can visualize how the wavepacket evolves over time. As the wavepacket evolves, it will:

1. Spread out due to dispersion (different energy components evolve at different rates)
2. Reflect off the box walls
3. Interfere with itself after reflections
4. Eventually undergo quantum revivals, where the wavepacket periodically returns close to its initial shape

The animation code below demonstrates this evolution:

```

xp = np.linspace(0, np.pi, 1000)

fig, ax = plt.subplots(1, 1, figsize=(10, 5))
plt.xlim(-0.1, np.pi+0.1)
plt.ylim(0, 0.02)
plt.axvline(x=0, ls='--', color='k')

```

```
plt.axvline(x=np.pi, ls='--', color='k')
plt.xlabel('Position x')
plt.ylabel(r'Probability density  $|\Psi(x,t)|^2$ ')
plt.title('Time Evolution of Wavepacket in a Box')
plt.tight_layout()
plt.draw()

background = fig.canvas.copy_from_bbox(ax.bbox)
points = ax.plot(xp, np.abs(w_func(xp, np.pi, c0n, 0))**2)[0]
plt.close()
```

```
## setup the canvas
canvas = Canvas(width=800, height=380, sync_image_data=False)
display(canvas)
```

```
for t in np.linspace(0, 4, 1000):
    fig.canvas.restore_region(background)
    ax.draw_artist(points)
    points.set_data(xp, 10*np.abs(w_func(xp, np.pi, c0n, t))**2)

    fig.canvas.blit(ax.bbox)
    X = np.array(fig.canvas.renderer.buffer_rgba())

    with hold_canvas(canvas):
        canvas.clear()
        canvas.put_image_data(X)
        sleep(0.02)
```

Quantum Revivals and Interference

The animation reveals several interesting quantum phenomena:

1. **Wavepacket Dispersion:** The wavepacket initially spreads out because different energy components evolve at different rates.
2. **Reflection and Interference:** When the wavepacket reaches the boundaries, it reflects and begins to interfere with itself.
3. **Quantum Revivals:** After some time, the wavepacket approximately returns to its initial shape. This is a purely quantum phenomenon called a “quantum revival.”

The revival time can be estimated as:

$$T_{\text{revival}} \approx \frac{4md^2}{\pi\hbar} \quad (11)$$

In our natural units ($\hbar = 1$, $m = 1$, $d = \pi$), this gives $T_{\text{revival}} \approx 4$.

Extensions and Further Exploration

Here are some interesting variations you could explore:

1. **Moving Wavepacket:** Modify the initial wavepacket to include non-zero momentum ($k_0 \neq 0$) and observe how it bounces back and forth between the walls.
2. **Different Initial Shapes:** Try different initial wavepacket shapes, such as a superposition of two Gaussians or a triangular wavepacket.
3. **Finite Potential Barriers:** Modify the simulation to have finite potential barriers, which would allow for tunneling.
4. **Time-Dependent Potential:** Explore what happens when the box width changes with time (like a piston), which can lead to quantum heating or cooling.
5. **Position Measurements:** Simulate the effect of position measurements during the time evolution, which would collapse the wavefunction.

To implement a moving wavepacket, simply set k_0 to a non-zero value in the `gauss_x` function, representing the initial momentum of the particle:

```
#| autorun: false
# Example: Gaussian wavepacket with initial momentum
xp = np.linspace(0, np.pi, 1000)

plt.figure(figsize=get_size(10, 6))
plt.plot(xp, np.abs(gauss_x(xp, 0.1, np.pi/4, 20))**2)
plt.xlabel('Position x')
plt.ylabel(r'Probability density  $|\Psi(x)|^2$ ')
plt.axvline(x=0, ls='--', color='k')
plt.axvline(x=np.pi, ls='--', color='k')
plt.tight_layout()
plt.show()
```

The real part of this wavefunction would show oscillations due to the momentum term e^{ik_0x} , representing the particle's motion.